

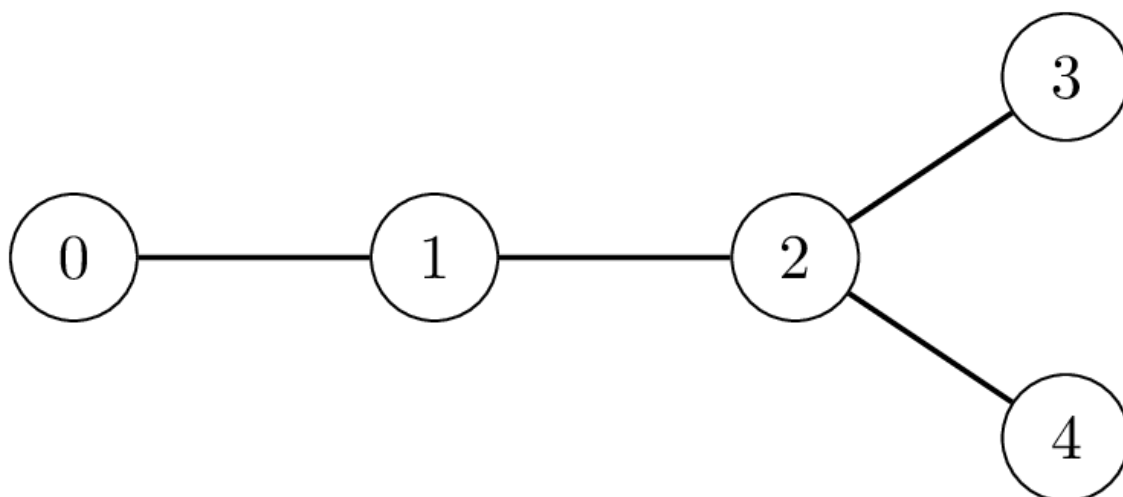
Migrations

Музей естественной истории исследует пути миграции динозавров в Боливии. Палеонтологи обнаружили следы динозавров на N различных участках, пронумерованных от 0 до $N - 1$ в порядке уменьшения возраста: на месте 0 находятся самые древние следы, а на месте $N - 1$ — самые поздние.

Динозавры мигрировали на каждый участок (кроме участка 0) с некоторого, более древнего участка. Для каждого участка i , такого что $1 \leq i \leq N - 1$, существует ровно один более древний участок $P[i]$ (где $P[i] < i$), такой что некоторые динозавры мигрировали непосредственно с участка $P[i]$ на участок i . Один более древний участок мог быть источником миграции для нескольких более поздних участков.

Палеонтологи в своей модели соединяют участки, между которыми осуществлялась миграция, **двусторонним переходом** между участками i и $P[i]$. Заметим, что для любых двух разных участков x и y можно перейти с x на y , пройдя некоторую последовательность переходов. **Расстояние** между участками x и y определяется как минимальное количество переходов, необходимое для перемещения с x на y .

Например, на следующем изображении показан случай с $N = 5$ участками, где $P[1] = 0$, $P[2] = 1$, $P[3] = 2$ и $P[4] = 2$. От участка 3 до участка 4 можно добраться через 2 перехода, поэтому расстояние между ними равно 2.



Задача музея — определить пару участков с максимальным расстоянием между ними. Обратите внимание, что ответ не обязательно будет единственным: например, в приведенном выше примере обе пары $(0, 3)$ и $(0, 4)$ имеют расстояние 3, которое является максимальным. В таком случае правильным ответом считается любая из пар на максимальном расстоянии.

Изначально значения $P[i]$ **неизвестны**. Музей посылает исследовательскую группу, которая по очереди посещает участки $1, 2, \dots, N - 1$, в этом порядке. Достигнув очередного участка i ($1 \leq i \leq N - 1$), команда выполняет два следующих действия:

- Определяет значение $P[i]$, то есть источник миграции на участок i .
- Принимает решение: отправить **одно** сообщение в Музей или **не отправлять** в Музей сообщение, основываясь на полученной к этому моменту информации.

Сообщения передаются через дорогостоящую систему спутниковой связи, поэтому каждое сообщение должно быть целым числом между 1 и 20 000 включительно. Кроме того, команде суммарно разрешается отправить **не более 50 сообщений**.

Ваша задача — реализовать стратегию, с помощью которой:

- Исследовательская группа выбирает участки, с которых будут отправляться сообщения, а также сами сообщения.
- Музей может определить пару участков с максимальным расстоянием, основываясь исключительно на полученных сообщениях, включая информацию о том, с каких участков они были отправлены.

Отправка большого количества сообщений через спутник требует больших затрат. Ваш балл будет зависеть как от наибольшего отправленного числа, так и от общего количества переданных сообщений.

Детали реализации

Вам нужно реализовать две функции: одну для исследовательской группы, другую — для музея.

Функция, которую вы должны реализовать для **исследовательской группы**, выглядит следующим образом:

```
int send_message(int N, int i, int Pi)
```

- N : количество участков.
- i : номер участка, который команда посещает в данный момент.
- Pi : значение $P[i]$.
- Эта функция вызывается $N - 1$ раз для каждого теста, в порядке $i = 1, 2, \dots, N - 1$.

Эта функция должна вернуть $S[i]$ с указанием действия, предпринятого исследовательской группой на участке i :

- $S[i] = 0$: исследовательская группа решает не отправлять сообщение с участка i .
- $1 \leq S[i] \leq 20\,000$: исследовательская группа отправляет целое число $S[i]$ в качестве сообщения с участка i .

Функция, которую вы должны реализовать для **Музея**, следующая:

```
std::pair<int,int> longest_path(std::vector<int> S)
```

- S : массив длины N такой, что:
 - $S[0] = 0$.
 - Для каждого $1 \leq i \leq N - 1$, $S[i]$ является целым числом, возвращаемым `send_message(N, i, P[i])`.
- Эта функция вызывается ровно один раз для каждого теста.

Эта функция должна возвращать пару участков (U, V) с максимальным расстоянием.

При оценке решения программа, вызывающая вышеуказанные функции, запускается **ровно два раза**.

- Во время первого запуска программы:
 - `send_message` вызывается ровно $N - 1$ раз.
 - **Ваша программа может сохранять и использовать вспомогательную информацию в глобальных переменных при последовательных вызовах `send_message`**
 - Возвращаемые значения (массив S) хранятся в грейдере.
 - В некоторых случаях поведение грейдера является **адаптивным**. Это означает, что значение $P[i]$ в вызове `send_message` может зависеть от действий, предпринятых исследовательской группой во время предыдущих вызовов.
- Во время второго запуска программы:
 - `longest_path` вызывается ровно один раз. Единственное, что передается в эту функцию от первого запуска — это массив S .

Ограничения

- $N = 10\,000$
- $0 \leq P[i] < i$ для каждого i , такого что $1 \leq i \leq N - 1$.

Подзадачи и баллы

Подзадача	Баллы	Дополнительные ограничения
1	30	Участок 0 и некоторый другой участок имеют максимальное расстояние среди всех пар участков.
2	70	Нет дополнительных ограничений.

Пусть Z — максимальное число, встречающееся в массиве S , а M — количество сообщений, отправленных исследовательской группой.

Если в любом из тестов выполняется хотя бы одно из следующих условий, то балл вашего решения для этого теста будет равен 0 (в CMS отображается как `Output isn't correct`):

- Хотя бы один элемент в S не удовлетворяет ограничениям.
- $Z > 20\,000$ или $M > 50$.
- Пара участков, которую вернула `longest_path` неверна.

В противном случае, балл за подзадачу 1 вычисляется следующим образом:

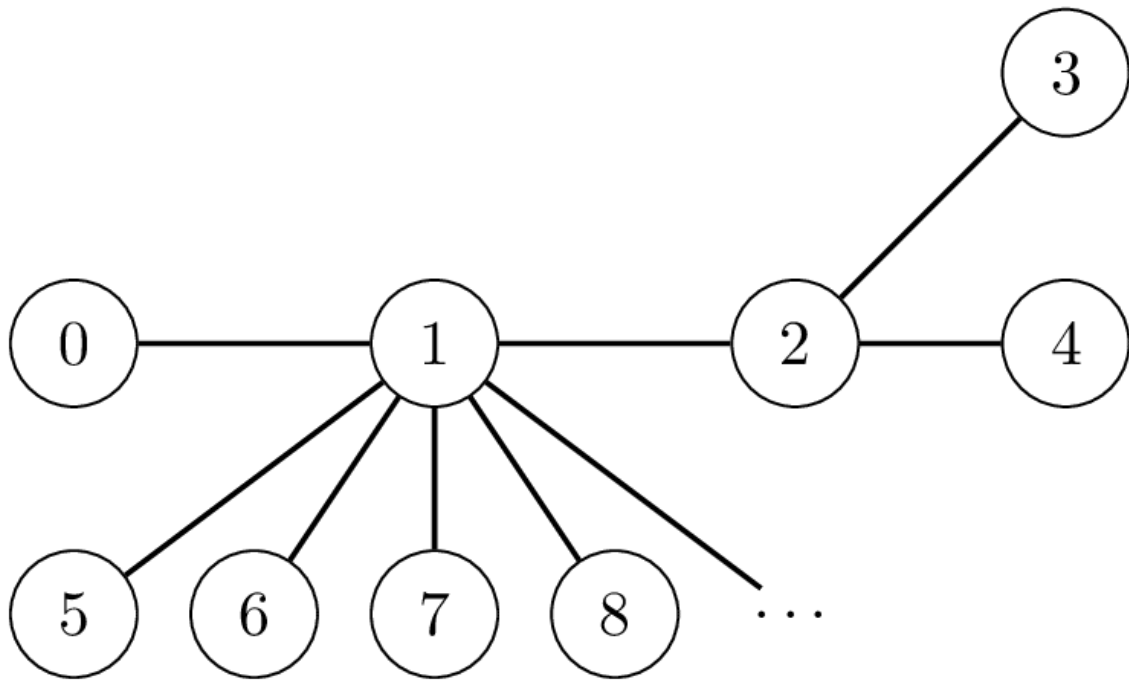
Условие	Баллы
$9\,998 \leq Z \leq 20\,000$	10
$102 \leq Z \leq 9997$	16
$5 \leq Z \leq 101$	23
$Z \leq 4$	30

Ваш балл за подзадачу 2 рассчитывается следующим образом:

Условие	Баллы
$5 \leq Z \leq 20\,000$ и $M \leq 50$	$35 - 25 \log_{4000} \left(\frac{Z}{5} \right)$
$Z \leq 4$ и $32 \leq M \leq 50$	40
$Z \leq 4$ и $9 \leq M \leq 31$	$70 - 30 \log_4 \left(\frac{M}{8} \right)$
$Z \leq 4$ и $M \leq 8$	70

Пример

Пусть $N = 10\,000$. Рассмотрим пример, в котором $P[1] = 0$, $P[2] = 1$, $P[3] = 2$, $P[4] = 2$ и $P[i] = 1$ для $i > 4$.



Предположим, что стратегия исследовательской группы состоит в том, чтобы отправлять сообщение $10 \cdot V + U$ всякий раз, когда пара участков (U, V) с максимальным расстоянием изменяется в результате вызова `send_message`.

Изначально парой с максимальным расстоянием является $(U, V) = (0, 0)$. Рассмотрим следующую последовательность вызовов во время первого запуска программы:

Вызов функции	(U, V)	Возвращаемое значение ($S[i]$)
<code>send_message(10000, 1, 0)</code>	$(0, 1)$	10
<code>send_message(10000, 2, 1)</code>	$(0, 2)$	20
<code>send_message(10000, 3, 2)</code>	$(0, 3)$	30
<code>send_message(10000, 4, 2)</code>	$(0, 3)$	0

Обратите внимание, что во всех оставшихся вызовах значение $P[i]$ равно 1. Это означает, что пара с максимальным расстоянием не меняется, и команда больше не отправляет никаких сообщений.

Затем, во втором запуске программы, выполняется следующий вызов:

```
longest_path([0, 10, 20, 30, 0, ...])
```

Музей читает последнее сообщение, отправленное исследовательской группой: $S[3] = 30$, и делает вывод, что $(0, 3)$ — это пара участков с максимальным расстоянием. Поэтому вызов возвращает $(0, 3)$.

Обратите внимание, что при таком подходе Музей не всегда может правильно определить пару с максимальным расстоянием.

Пример грейдера

Пример грейдера вызывает обе функции `send_message` и `longest_path` в одном и том же запуске, что отличается от реального грейдера.

Формат ввода:

```
N
P[1] P[2] ... P[N-1]
```

Формат вывода:

```
S[1] S[2] ... S[N-1]
U V
```

Обратите внимание, что вы можете использовать пример грейдера с любым значением N .